DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

A Library C Can Call

CMSC 131 Laboratory Activity 2

Prepared by: Rene Andre Bedonia Jocsing

Published: October 16, 2026 **Deadline:** December 09, 2026

This laboratory activity is about subprograms. You'll build `renlib`, a library of sorting and searching routines written entirely in NASM, callable from C, and benchmarked against the C standard library's own versions.

Background

Everything you've written so far ran top to bottom in one piece. Activity 1 had three routines. One of them, `encode_header`, called `ip_checksum` once and got a number back. Nothing called itself.

Recursion breaks that. A routine that calls itself needs somewhere to keep each call's own copy of its variables. The caller's values must survive until the callee returns. Six registers can't hold an unknown number of pending calls.

The answer is the **stack**, a region of memory that grows downward, and a discipline for using it called a **calling convention**. Get the discipline right and recursion works, C can call your code, and your code can call C's. Get it wrong and the failure appears somewhere unrelated, usually long after the mistake.

That discipline is the subject here. Sorting is the excuse to exercise it. Quicksort is naturally recursive and hard to get wrong in a way that still sorts, which makes it a fair test of the machinery underneath.

Learning Objectives

By the end of this laboratory activity, students should be able to:

- Set up and tear down a stack frame with a correct prologue and epilogue
- Pass arguments and return values under the cdecl calling convention
- Identify which registers a callee must preserve and preserve them
- Implement a recursive algorithm whose depth isn't known in advance
- Explain what makes a subprogram reentrant, and demonstrate it
- Split a library across multiple assembled modules that link together
- Measure performance honestly and report what the measurement shows

---

Task

Build four C-callable assembly routines, split across three modules, and link them into `bench`, the provided C harness:

- `qsort_asm`, a recursive quicksort over an array of 32-bit signed integers
- `bsearch_asm`, a recursive binary search over a sorted array
- `gcd_asm`, a recursive Euclid's algorithm over nonnegative integers, and the simplest proof your frames work
- `sum_range_asm`, a demonstrably reentrant routine described below

The four are linked with `bench.c` into one program. Nothing is packaged as a separate library file. Nothing loads at run time.

Required Features

The C Interface

The provided `bench.c` declares your routines and calls them. You implement them.

```c
/* Sorts n ints in place, ascending. */
void PRE_CDECL qsort_asm(int *arr, int n) POST_CDECL;

/* Returns the index of key in the sorted array, or -1 if absent. */
int PRE_CDECL bsearch_asm(int *arr, int n, int key) POST_CDECL;

/* Returns the greatest common divisor of a and b, both nonnegative. */
int PRE_CDECL gcd_asm(int a, int b) POST_CDECL;

/* Returns the sum of the integers from lo to hi inclusive, when that sum
 * fits a signed 32-bit integer. */
int PRE_CDECL sum_range_asm(int lo, int hi) POST_CDECL;

/* Set by gcd_asm: the deepest level its recursion reached. */
extern int gcd_depth;

/* Called by sum_range_asm once per call. bench.c owns it. */
void bench_callback(void);
```

`qsort_asm` takes no comparison function. C's `qsort` is called with one, `cmp_int` in `bench.c`, and it's the only comparison implementation the harness supplies. Your sort compares values itself, in a register.

`gcd_asm` is defined for nonnegative operands. `gcd(a, 0)` is `a`, `gcd(0, b)` is `b`, and `gcd(0, 0)` is 0. `sum_range_asm` is defined whenever the sum fits a signed 32-bit integer. Outside that range the result is undefined. The harness never asks for one.

Two more identifiers cross the boundary, in the other direction. `gcd_depth` is a global your assembly defines in `.bss` and exports, and `bench.c` prints it after `--gcd`. It holds the recursion depth of the most recent `gcd_asm` call, meaning the number of `gcd` calls on the stack at the deepest point, with the first call counted as 1. `bench_callback` is a C function your `sum_range_asm` calls once, every time C calls it. The reentrancy check below runs on that call.

Running it

```
./bench --sort tests/random1000.txt
```

```
Read 1000 integers.
qsort_asm:   sorted, verified ascending
C qsort:     sorted, verified ascending
Match:       YES
```

```
qsort_asm:   0.019 ms per sort (mean of 200 runs)
C qsort:     0.042 ms per sort (mean of 200 runs)
Ratio:       0.44x
```

Those timings are the reference implementation's, on one machine. Yours will differ. Sorting a thousand integers once is too quick for the clock to time, so `bench` runs each sort two hundred times from a fresh copy and reports the mean.

```
./bench --gcd 1071 462
```

```
gcd_asm(1071, 462) = 21
Recursion depth reached: 4
```

The `cdecl` *Convention*

This is the contract. Both sides obey it even if neither checks.

The caller pushes arguments right to left, then executes `call`, which pushes the return address. After the call returns, the caller removes the arguments.

The callee finds its arguments above `ebp` once the standard prologue has run:

```
qsort_asm:
        enter   0,0               ; push ebp; mov ebp, esp
        ; [ebp+0]  = saved ebp
        ; [ebp+4]  = return address
        ; [ebp+8]  = first argument
        ; [ebp+12] = second argument
        ...
        leave                     ; mov esp, ebp; pop ebp
        ret
```

`enter 0,0` and `leave` are the compact forms. The first pushes `ebp` and copies `esp` into it. The second reverses that exactly.

Registers split into two groups. `eax`, `ecx`, and `edx` are caller-saved, meaning you may destroy them freely. `ebx`, `esi`, `edi`, and `ebp` are callee-saved, meaning if you use them you must restore them before returning.

```
        push    ebx               ; save what you intend to use
        push    esi
        ...
        pop     esi               ; restore in reverse order
        pop     ebx
```

Return values go in `eax`.

Why This Matters More Here Than in Activity 1

In Activity 1, a clobbered `ebx` broke the C driver. The failure showed up somewhere in `driver.c`, after your routine had returned. That was one caller, one call deep. Here the routine that clobbers a register is also the routine that needed it.

A recursive routine is its own caller. If `qsort_asm` destroys `esi` and then calls itself, the outer invocation's `esi` is gone. That's a problem. The outer invocation is the one holding your array bounds. The symptom is a sort that works for two elements and corrupts memory for twenty. It's miserable to debug, so the discipline isn't optional.

Quicksort

Sort in place, ascending. The recursive structure:

```
qsort(arr, lo, hi):
    if lo >= hi: return
    p = partition(arr, lo, hi)
    qsort(arr, lo, p - 1)
    qsort(arr, p + 1, hi)
```

Partition with the Lomuto scheme, which is the easier one to get right in assembly:

```
partition(arr, lo, hi):
    pivot = arr[hi]
    i = lo - 1
    for j = lo to hi - 1:
        if arr[j] <= pivot:
            i = i + 1
            swap arr[i], arr[j]
    swap arr[i + 1], arr[hi]
    return i + 1
```

The C-facing `qsort_asm(arr, n)` calls an internal recursive routine with explicit `lo` and `hi`. Only the outer one has to obey cdecl. The inner one is yours and may use any convention you document.

Binary Search

Recursive, returning the index or `-1`:

```
bsearch(arr, lo, hi, key):
    if lo > hi: return -1
    mid = lo + (hi - lo) / 2
    if arr[mid] == key: return mid
    if arr[mid] < key:  return bsearch(arr, mid + 1, hi, key)
    else:               return bsearch(arr, lo, mid - 1, key)
```

Compute Mid That Way, Not the Obvious Way

`(lo + hi) / 2` is the obvious form and it's *wrong*. With large indices the sum overflows 32 bits and the midpoint lands outside the array. `lo + (hi - lo) / 2` can't overflow because `hi - lo` is never larger than the array. Use the safe form and say why in a comment.

No test file can catch the naive form. The sum only overflows once `hi` reaches 2^30. That's an array of over a billion integers, four gigabytes of them, and `bench` is a 32-bit program with four gigabytes of address space in total. Here the comment is the check. The defense asks for the reasoning behind the safe form.

Reentrancy

`sum_range_asm(lo, hi)` returns the sum of the integers from `lo` to `hi` inclusive, when that sum fits a signed 32-bit integer, and must be **reentrant**: correct when a second call begins before the first has finished.

A routine is reentrant when it holds no mutable state shared between invocations. Everything a call needs lives in its registers and on its stack, so a second call that begins mid-flight gets its own copy and can't disturb the first. Anything in `.data` or `.bss` that changes during execution breaks it, because two overlapping calls then write to the same variable.

Prove it. Your `sum_range_asm` calls `bench_callback` once per call, and `bench.c` uses that callback to start a second `sum_range_asm` before the first has finished. It checks both answers. Write the routine so it passes, and explain in the Design Notes section of `README.md` which of your design choices made it reentrant.

Recursion and reentrancy are different properties. Recursion doesn't imply reentrancy. A recursive routine that updates one static variable at every level is still recursive. Two overlapping calls would corrupt each other. Here the overlap is explicit because the callback starts the second call while the first is still on the stack. What would a non-reentrant version of this routine look like, and where exactly would it go wrong?

Technical Requirements

Getting the Starter

Fork cmsc-131-lab2-starter on GitHub and clone your fork. A fork is a snapshot. A fix pushed to the starter mid-week never reaches it, so the fixtures can't change under you.

Run these from Git Bash on Windows, the shell Block 1 set up, where `make` is your alias for `mingw32-make`. On Linux use your terminal.

```
git clone https://github.com/<your-account>/cmsc-131-lab2-starter.git
cd cmsc-131-lab2-starter
make
make check
```

The assembly files ship as stubs that assemble and link as-is, so the build works before any code is written. `make check` then fails 19 of its 36 checks. The seventeen that pass prove little. Four are files that arrive empty, single-element, or already sorted, which a sort that does nothing leaves alone. `gcd(0, 0)` expects 0, which a stub returns by accident. The hostile caller can't be upset by a stub that touches no register. The rest are refusals that the provided harness performs before any assembly runs. That red run is the correct starting state.

The provided files in the tree below are fixtures. The grader compares your fork against the starter, so an edited `bench.c`, `Makefile`, `run_tests.sh`, or `tests/` file shows up as a diff in the open. One member forks the repository under their account, keeps it public, and adds the rest of the group as collaborators, since that fork is the repository you submit.

Project Structure

```
renlib/
  Makefile              provided
  bench.c               provided: file I/O, timing, C-side comparison, hostile caller
  cdecl.h               provided
  LICENSE               provided
  sort.asm              yours: qsort_asm and its partition
```

```
  search.asm            yours: bsearch_asm
  util.asm              yours: gcd_asm, sum_range_asm
  tests/
    random1000.txt      provided
    sorted1000.txt      provided: already ascending, the worst case
    reverse1000.txt     provided: descending
    duplicates.txt      provided: 500 values drawn from five
    identical.txt       provided: 100 copies of one value
    extremes.txt        provided: zero, INT_MIN, and INT_MAX, each more than once
    tiny.txt            provided: two elements, reversed
    tiny1.txt           provided: one element
    tiny0.txt           provided: empty
    bad/
      word.txt          provided: a valid prefix, then a word
      suffix.txt        provided: a valid prefix, then "30z"
  run_tests.sh          provided
  README.md             yours: design notes, subsystem ownership, quirks and issues
```

The starter's `README.md` ends with three sections that ship with prompts and empty tables or lists under them: Design Notes, Subsystem Ownership, and Quirks and Issues. Design Notes and Subsystem Ownership are Week 1 deliverables under the syllabus. Quirks and Issues is Week 3's. The rest of the README is the repository's own notes on the gate and the fixtures. Leave those alone.

Three separate `.asm` files, assembled separately and linked together. Putting everything in one file doesn't meet the multi-module requirement.

Constraints

- Whole-number arithmetic only. No floating point, no FPU.
- `qsort_asm` and `bsearch_asm` must be recursive. An iterative implementation with an explicit stack array doesn't satisfy this activity.
- No calls into the C standard library from your assembly.
- Callee-saved registers must be preserved. `bench --hostile` checks this with a deliberately hostile caller, and the defense repeats the check by hand.

Implementation Notes

Indexing an Array of Integers

Elements are four bytes, so use the scale from Block 7:

```
; esi = base, ebx = index, load arr[ebx] into eax
        mov     eax, [esi + ebx*4]
```

Swapping two elements needs a temporary, since there's no memory-to-memory `mov`:

```
        mov     eax, [esi + ebx*4]
        mov     edx, [esi + ecx*4]
        mov     [esi + ebx*4], edx
        mov     [esi + ecx*4], eax
```

Recursing

The inner call needs its own arguments. Push them, call, clean up:

```
        push    edx             ; hi
        push    eax             ; lo
        push    esi             ; array base
        call    qsort_inner
        add     esp, 12         ; 3 arguments x 4 bytes
```

That `add esp, 12` is required. If you omit it, the three arguments are still on the stack when your epilogue runs. `leave` discards them if you kept a frame, but any `pop` before it takes an argument instead of the register you saved. The outer call then reads its array base from a register holding `lo`. The sort writes to memory it doesn't own.

Measuring

`bench.c` handles timing. Your job is to run it, record what it measures, and write the numbers into the Design Notes section of `README.md`.

Don't read much into a win on `random1000.txt`. C's `qsort` calls a comparison function through a pointer for every pair it looks at, while your assembly compares in a register, so a plain quicksort beats it there. The reference implementation does. The other files are where the ratio says something about your sort rather than about a function call.

What's expected is an honest measurement and an explanation you can defend. Report:

- The ratio on `random1000.txt`, `sorted1000.txt`, `reverse1000.txt`, `duplicates.txt`, and `identical.txt`
- Which input is your worst case, and why
- One specific change that would narrow the gap, and your estimate of by how much

Sorted Input Is the Interesting Case

Lomuto partitioning with the last element as pivot degrades to $O(n^2)$ on already-sorted input. Every partition puts one element on one side and everything else on the other. `sorted1000.txt` exists to make you see this happen. The reference measures a ratio in the thirties there, against under 1x on random input.

Report the ratio you measure there and explain the cause. You may fix it, by choosing a median-of-three pivot, and if you do, report both sets of numbers. Fixing it isn't required. Understanding the cause is.

Common Pitfalls

- Clobbering `ebx`, `esi`, `edi`, or `ebp` without saving them
- Forgetting to clean arguments off the stack after a `call`
- Using `(lo + hi) / 2` for the midpoint
- Recursing on `p` rather than `p - 1` and `p + 1`, which repeats the same range until the stack overflows and the program faults
- Comparing with unsigned jumps, which breaks the moment the array holds a negative number
- Clearing `edx` with `mov edx, 0` before `idiv`. That's right only when the dividend can't be negative. `cdq` is the sign extension `idiv` expects.
- Static storage inside `sum_range_asm`, which breaks reentrancy
- Assuming an empty or single-element array never reaches your routine

---

Testing Strategy

Correctness Against a Reference

`bench.c` sorts the same input with your routine and with C's `qsort`, then compares element by element. This is the oracle. It doesn't care how you sorted.

```
make check
```

`check` builds `bench`, then runs `run_tests.sh`, which runs every mode. Each run is captured first. Its exit status is read before its text is examined, so a program that prints the right line and then crashes still fails.

- `--sort` on every file in `tests/`, which must report `Match: YES`
- `--search` on every file, which must report `all probes correct`. The fixture is sorted with C's `qsort` first, so a failure on this path belongs to `bsearch_asm` alone.
- `--gcd 1071 462`, which must print 21 at depth 4, plus `--gcd 0 42`, `--gcd 42 0`, and `--gcd 0 0`, which must print 42, 42, and 0
- `--reentrancy` and `--hostile`, which must both print `PASS`
- the malformed arguments (`--gcd -1 5`, `--gcd 5x 7`, a missing operand) and the malformed files in `tests/bad/`, which must be refused with a nonzero status rather than read as smaller tests

(`make test` still works, as an alias for `check`.)

Edge Cases

- An empty array, and one of a single element (`tiny0.txt`, `tiny1.txt`)
- Two elements, already ordered and reversed (`tiny.txt` is the reversed pair, and the ordered one is yours to add)
- An array where every element is identical (`identical.txt`)
- An array containing negative numbers and zero (`extremes.txt`, which also holds the largest and smallest 32-bit values)
- A key absent from the array, returning `-1` (`--search` probes one below everything, one above, and one in a gap)
- A key at the very first and very last positions (`--search` probes both)
- A file that isn't a list of integers (`tests/bad/word.txt` carries a word after a valid prefix, and `tests/bad/suffix.txt` carries `30z`). `bench` refuses the whole file rather than reading the valid prefix, and exits nonzero.

Reentrancy

```
./bench --reentrancy
```

Both the outer and inner results must be correct. The callback must have run exactly twice, once for each call from C, so a recursion that calls it at every level fails. A `sum_range_asm` that never calls `bench_callback` can't be checked, so `bench` fails it. A wrong result means static state somewhere.

The Hostile Caller

```
./bench --hostile
```

`bench` loads a sentinel into `ebx`, `esi`, and `edi`, calls each of your four routines, and reads the registers back. It checks `esp` the same way. A routine that clobbers one is named in the output. The stubs pass this check because they touch nothing, so a pass on its own proves little.

Deliverables

Your group's GitHub repository, with a clean commit history showing individual contributions, must contain:

1. `sort.asm`, `search.asm`, and `util.asm`, documented
2. The provided `Makefile`, `bench.c`, `run_tests.sh`, and `tests/`, unmodified, plus any test files you added
3. The Design Notes section of `README.md`, covering:
   - A stack frame diagram for one recursive call of `qsort_asm`, showing where arguments, the return address, saved `ebp`, and saved registers sit
   - Which registers you preserve and why
   - Measured ratios on `random1000.txt`, `sorted1000.txt`, `reverse1000.txt`, `duplicates.txt`, and `identical.txt`
   - Your worst case, with the cause explained
   - One improvement, with an estimated effect
   - Why `sum_range_asm` is reentrant
4. The Subsystem Ownership section of `README.md`, naming which member owns which subsystem, and the Quirks and Issues section, filled in by Week 3

Subsystem Ownership

Subsystem	Covers
Sorting	`qsort_asm`, partition, the recursive driver
Searching	`bsearch_asm`, `gcd_asm`, index arithmetic
Convention and measurement	cdecl compliance, register preservation, `sum_range_asm`, benchmarks, the Design Notes

Record the split in the Subsystem Ownership section of `README.md` before the Week 1 progress report. You'll be asked at defense about a subsystem you didn't write.

To submit, one designated member emails the group submission, sending the link to your group's public GitHub repository. CC the rest of your group. The commit you defend is the one that's graded, so push before your defense slot. Your provisional grade comes from that commit. The syllabus then gives you one week to commit any refactors your instructor requires at the defense. Those commits can raise the grade. Nothing else you push after the defense changes it. Adhere to the following subject line, joining every group member's name with `&` in the same `LastName, Initials` format: `[CMSC 131 Lab] Lab 2 Group: LastName1, Initials1 & LastName2, Initials2 & LastName3, Initials3`. A quadro adds a fourth name the same way.

Then, each member submits, individually, through email:

1. A short `reflection.txt` covering what you built on your subsystem, what fought back, and what you learned.
2. A short `peer.txt` with your honest assessment of how your other groupmates worked during the activity.

Adhere to the following subject line: `[CMSC 131 Lab] Lab 2: Surname, Initials`.

For example: `[CMSC 131 Lab] Lab 2 Group: Sanchez, SM & Reyes, JD & Cruz, AB` for the group submission and `[CMSC 131 Lab] Lab 2: Sanchez, SM` for the individual submission.

Progress Reports

The syllabus says what every progress report expects in general terms: design notes, the subsystem split, and a working prototype in Week 1, a functional prototype with the core features in Week 2, and

the complete implementation with test results and documented quirks in Week 3. This section says what those mean for this activity. Each report is a 15-minute slot. Come with the repository open and `bench` built, and expect every member to speak.

Week 1

The interface, and a plan. This activity opens while Lab 1 is in its defense window, so the first week is about reading `bench.c` before writing assembly against it. Bring:

- The Subsystem Ownership section of `README.md`, filled in, and a commit history that already shows who owns what
- The Design Notes section of `README.md` started: the four routines `bench.c` expects and what each one returns, and a timeline naming the subsystem each week finishes
- A prototype that builds and runs `bench` on a provided test file, whatever it prints today
- How the caller passes arguments to your routines, who removes them from the stack afterwards, and which registers you must preserve
- Why the sort and the search must be recursive here, and why three assembly files rather than one

Week 2

The sort and the search, working. Bring:

- `--sort` on a provided file reporting `Match: YES`, including `extremes.txt`, which holds negative numbers and zero
- Your partition routine, what it returns, and the range each of the two recursive calls receives
- Your midpoint calculation in `bsearch_asm`, and `-1` for a key that isn't there
- Where you save and restore the callee-saved registers
- A commit history spread across the week. Each member opens code they wrote and explains a routine inside it

Week 3

Everything green, and the numbers. Bring:

- `make check` passing all 36 checks, including `--reentrancy` and `--hostile` printing `PASS`
- The design choice that made `sum_range_asm` reentrant, and what a non-reentrant version would have looked like
- The Design Notes section of `README.md` complete: the stack frame diagram, the measured ratios on the five files the Measuring section names, your worst case with its cause, and one improvement with an estimated effect
- The Quirks and Issues section of `README.md`, filled in, with at least one entry you can explain
- The tests you added beyond the provided cases, the provided files unmodified, and three weeks of commits distributed across every member

Week 4

The defense. It takes a 25-minute slot. Your instructor runs `bench` against its reference and questions each member on a subsystem they didn't write. The commit you defend is the one that's graded.

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, DeepSeek, etc.) to generate wholesale vibe-coded or vibe-written work is considered cheating. Cheating is against the university's code of ethics. It's subject to failure in the course, harsh disciplinary action, or expulsion. The syllabus's Academic Integrity and Honesty section is the policy authority. It applies in full.

Important Dates

This activity has no deadline of its own. It has a prescribed timeline, the four windows below, and the syllabus attaches extra credit to keeping it. The defense belongs in the Week 4 window. The one hard cutoff is course-wide: after the end of classes on December 9, no laboratory deliverable is accepted for any activity.

Progress reports and laboratory defense may be booked only during the hours defined in the syllabus, with your own laboratory instructor, on that instructor's booking page. The syllabus lists one page per instructor. Book ahead.

Activity	Window	Sec 3 (W)	Sec 1 & 2 (T/F)
Week 1 Progress Report	Oct 19-25	Oct 21	Oct 20, 23
Week 2 Progress Report	Oct 26-Nov 1	Oct 28	Oct 27, 30
Week 3 Progress Report	Nov 2-8	Nov 4	Nov 3, 6
Defense Window	Nov 9-15	Nov 11	Nov 10, 13

Grading Rubric (100 Pts)

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
System Architecture (25%)	Genuine multi-module split. Clean separation between the cdecl boundary and internal routines. Frame layout deliberate and documented.	Mostly modular. The C-facing and internal conventions blur in places.	One large file or one large routine. Recursion structure hard to follow.	No modularity. The calling convention is violated or ignored.
Robustness (20%)	Correct on every edge case including empty, single-element, all-duplicate, and negative-valued arrays. Every call leaves the stack and the callee-saved registers as it found them.	Core cases correct. Minor issues on duplicates or boundary indices.	Sorts typical input but fails an edge case or leaves an argument on the stack after a call.	Fails the provided tests or corrupts memory.
Code Engineering (10%)	Callee-saved registers preserved without exception. Stack cleaned after every call. Comments explain frame layout and the midpoint formula.	Registers preserved. A few unexplained choices.	Inconsistent preservation. Stack cleanup sometimes missing.	Clobbers registers the caller needs. Arguments are left on the stack.
Collaboration (20%)	Atomic semantic commits distributed across the month. Clear per-member ownership. Evidence of real code review between members.	Consistent version control. Adequate messages. Work visibly shared.	Inconsistent Git usage. Large dumps instead of increments. Vague messages.	Minimal history. No evidence of the whole group contributing.
Technical Defense (25%)	Every member traces a recursive call through the stack by hand. Each explains a subsystem they didn't write. Benchmark claims are defended with evidence.	Clear explanation. All members participate. Measurements understood.	Uneven participation. Struggles to draw the frame or explain the worst case.	Can't explain the convention or the recursion. Benchmarks unexamined.